SECTION A: Conceptual Answers

1. Spring MVC Request Flow (JSP → Controller → DB → View)

Browser submits JSP form
↓
DispatcherServlet (Front Controller)
↓
HandlerMapping (finds the right Controller)
↓
Controller method executes
↓
DAO / JdbcTemplate queries DB
↓
Result added to Model
↓
ViewResolver maps logical name → JSP
↓
JSP rendered and sent back to browser

2. IoC and Dependency Injection

- **IoC (Inversion of Control):** Instead of your class creating its own dependencies (new UserDAO()), the Spring container creates and manages them for you. Control is *inverted* — the framework controls object creation.
- **DI (Dependency Injection):** The mechanism Spring uses to inject dependencies into a class, either via constructor or setter.

java
```
// Without IoC — tightly coupled
public class LoginService {
   UserDAO dao = new UserDAO(); // you manage it
}

// With DI — loosely coupled
public class LoginService {
   @Autowired
   UserDAO dao; // Spring injects it
}
```

3. BeanFactory vs ApplicationContext

Feature	BeanFactory	ApplicationContext
Bean creation	Lazy (on demand)	Eager (at startup)
Annotation support	❌	✅

Feature	BeanFactory	ApplicationContext
Event publishing	❌	✅
I18n (messages)	❌	✅
Use case	Lightweight/legacy	All modern Spring apps

Always use **ApplicationContext** in real projects.

4. How JdbcTemplate Simplifies JDBC

Without JdbcTemplate you need to: get connection → create statement → execute → handle ResultSet → close everything → catch exceptions everywhere.

JdbcTemplate handles all of that. You only write the SQL and row mapping:

java

// Raw JDBC = ~20 lines

// JdbcTemplate = 2 lines:

String sql = "SELECT * FROM users WHERE email = ?";

User user = jdbcTemplate.queryForObject(sql, new BeanPropertyRowMapper<>(User.class), email);

5. Role of DispatcherServlet

It is the **Front Controller** — a single entry point for all HTTP requests in a Spring MVC app. It:

- Receives every request
- Delegates to the right controller via HandlerMapping
- Gets the ModelAndView back
- Passes it to ViewResolver
- Returns the final response

6. Constructor vs Setter Injection

```
java
// Constructor Injection — dependency is REQUIRED
@Component
public class LoginService {
   private final UserDAO userDAO;

   @Autowired
   public LoginService(UserDAO userDAO) {
      this.userDAO = userDAO; // immutable, preferred
   }
}
```

```
// Setter Injection — dependency is OPTIONAL
@Component
public class LoginService {
   private UserDAO userDAO;

   @Autowired
   public void setUserDAO(UserDAO userDAO) {
      this.userDAO = userDAO;
   }
}
```

	Constructor	Setter
Dependency required?	Yes	No (optional)
Immutability	✅	❌
Preferred by Spring team	✅	Only for optional deps